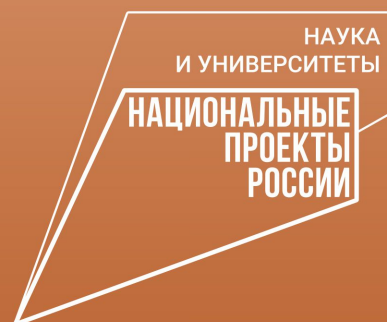




Казанский
федеральный
университет

СУНЦ КФУ
IT-лицей



АЛГОРИТМИЧЕСКИЕ МЕТОДЫ СЕГМЕНТАЦИИ ИЗОБРАЖЕНИЙ

МЕТОДИЧЕСКАЯ БИБЛИОТЕКА

КАЗАНСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ
Специализированный учебный научный центр-
общеобразовательная школа-интернат «IT-лицей»

АЛГОРИТМИЧЕСКИЕ МЕТОДЫ СЕГМЕНТАЦИИ ИЗОБРАЖЕНИЙ

Учебное пособие



КАЗАНЬ
2023

УДК 004.93(075)
ББК 32.81я7
А45

*Печатается по рекомендации
Ученого совета Специализированного учебного научного центра –
общеобразовательной школы-интерната «ИТ-лицей»
Казанского (Приволжского) федерального университета
(протокол № 2 от 15 октября 2023 г.)*

Авторы:

заместитель директора по научной деятельности
Института вычислительной математики и информационных технологий КФУ,
кандидат физико-математических наук, доцент **Д.Н. Тумаков**;
лаборант Института геологии и нефтегазовых технологий **З.Д. Каюмов**;
кандидат технических наук, директор Института вычислительной математики
и информационных технологий КФУ **А.А. Егорчев**;
учитель информатики высшей квалификационной категории
СУНЦ ИТ-лицей КФУ **И.И. Латыпов**

Научный редактор

кандидат физико-математических наук, доцент **Ф.М. Гафаров**

Рецензенты:

доктор физико-математических наук, профессор **Н.Б. Плещинский**;
кандидат физико-математических наук, доцент **А.А. Колчев**

А45 **Алгоритмические методы сегментации изображений** [Электронный ресурс]: учебное пособие / Д.Н. Тумаков, З.Д. Каюмов, А.А. Егорчев и др. – Электронные текстовые данные (1 файл: 2 Мб). – Казань: Издательство Казанского университета, 2023. – 40 с. – Системные требования: Adobe Acrobat Reader. – URL: https://kpfu.ru/portal/docs/F265768322/Algoritmicheskie.metody.segmentacii.izobra.zhenii_.pdf. – Загл. с титул. экрана.

ISBN 978-5-00130-773-0

В пособии описаны основные методы и алгоритмы выделения объектов и их границ на изображениях. Подробно рассмотрены классические методы сегментации. Приведены примеры программ на языке Python. Показаны результаты сегментации различными алгоритмами на макроснимках зерен горных пород. В конце каждой главы имеются контрольные вопросы и задания для лабораторных работ.

Пособие предназначено для обучающихся, осваивающих программу среднего общего образования по информатике на углубленном уровне, а также для студентов ИТ-направлений.

УДК 004.93(075)
ББК 32.81я7

ISBN 978-5-00130-773-0

© Издательство Казанского университета, 2023

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	4
ГЛАВА 1. ПОРОГОВАЯ СЕГМЕНТАЦИЯ	6
Пороговая бинаризация	6
Адаптивная бинаризация	7
Контрольные вопросы	9
Лабораторная работа	9
ГЛАВА 2. МЕТОДЫ НА ОСНОВЕ РЕГИОНОВ	10
Метод сегментации Region Growing	10
Метод сегментации изображений Split-and-Merge	11
Сегментация методом водораздела	12
Контрольные вопросы	19
Лабораторная работа	20
ГЛАВА 3. МЕТОДЫ СЕГМЕНТАЦИИ ИЗОБРАЖЕНИЙ НА ОСНОВЕ ГРАНИЦ	21
Оператор Собеля	21
Оператор Кэнни	23
Контрольные вопросы	25
Лабораторная работа	25
ГЛАВА 4. МЕТОДЫ НА ОСНОВЕ КЛАСТЕРИЗАЦИИ	26
K-Means Clustering	26
Mean Shift	28
DBSCAN	32
Контрольные вопросы	34
Лабораторная работа	34
ГЛАВА 5. СЕГМЕНТАЦИЯ НА ОСНОВЕ АКТИВНОГО КОНТУРА	35
Контрольные вопросы	38
Лабораторная работа	38
ЛИТЕРАТУРА	39

ВВЕДЕНИЕ

Сегментация изображений является одним из важных этапов обработки изображений. Сегментация позволяет выделить на изображении объекты и их границы. Сегментация используется во многих областях, таких как компьютерное зрение, медицинская диагностика, автоматическое распознавание образов и т.д.

В данном учебном пособии рассмотрены различные методы сегментации изображений, прежде всего такие классические методы, как пороговая сегментация и кластерный анализ. Изложен также современный подход – сегментация с помощью сверточных нейронных сетей. Выделены основные проблемы, с которыми можно столкнуться при проведении сегментации изображений, такие как шумы, размытие, слабые контрасты и другие, предложены методы устранения этих проблем.

Все практические примеры представлены на языке программирования Python, который широко используется в области обработки изображений. Приведены краткие описания некоторых библиотечных функций. Для операций с изображениями использованы следующие библиотеки.

1. *NumPy*: Библиотека для работы с многомерными массивами данных. Эта библиотека позволяет эффективно выполнять математические операции и манипуляции с данными.
2. *OpenCV*: Open Source Computer Vision Library – мощный инструмент для обработки изображений и видео. Предоставляет набор функций для чтения, обработки и анализа визуальных данных.
3. *Scikit-image*: Библиотека, специализирующаяся на обработке изображений. Содержит разнообразные методы для выполнения операций, таких как фильтрация, морфологические операции и сегментация.
4. *Matplotlib*: Библиотека для создания визуализаций и графиков. Используется для визуализации изображений и результатов сегментации.

Сегментация изображений – это процесс выделения на изображении нескольких сегментов или областей, которые содержат объекты с определенными свойствами. В пособии рассматриваются следующие наиболее распространенные методы сегментации изображений.

1. *Пороговая сегментация* – это простой метод, который основывается на выборе порогового значения для интенсивности или цвета пикселей, чтобы разделить изображение на две или более областей. Global Thresholding, Adaptive Thresholding, Otsu's Thresholding.
2. *Методы на основе регионов* – это методы, которые сегментируют изображение, объединяя соседние пиксели со схожими свойствами (в один регион). Region Growing, Split-and-Merge, Watershed.
3. *Методы сегментации на основе границ* – это методы, которые используют границы объектов на изображении для выделения различных областей. Оператор Собеля, Оператор Кэнни.
4. *Методы на основе кластеризации* – это методы, которые используют алгоритмы кластеризации для группировки пикселей в кластеры схожих свойств. K-Means Clustering, Mean Shift, DBSCAN.
5. *Сегментация на основе активного контура* – метод, который использует кривые, называемые контурами, для выделения объектов на изображении. Активный контур – это кривая, которая «активно» перемещается на изображении в направлении границы объекта и следует за контурами объекта, пока не будет полностью его охватывать. Snakes, Active Contour Model (ACM), Gradient Vector Flow (GVF).

На сегодняшний день широко используются также методы на основе глубокого обучения – это методы, в которых применяются сверточные нейронные сети для сегментации изображений. Обучение нейронных сетей происходит на больших наборах данных с размеченными изображениями. Например, такие сети, как U-Net, Mask R-CNN и Panoptic FPN, наиболее часто используются для сегментации. Используются и методы, основанные на использовании графа для представления изображения. В этих методах находится минимальный разрез в графе, что позволяет разделить изображение на несколько областей: Normalized Cut, Max-Flow/Min-Cut, Random Walks. Эти методы условно носят название сегментации при помощи разрезов на графах. Методы с использованием нейронных сетей и графов выходят за рамки настоящего пособия.

После освоения материала, представленного в пособии, вы сможете эффективно проводить сегментацию отдельных элементов на изображениях.

ГЛАВА 1

ПОРОГОВАЯ СЕГМЕНТАЦИЯ

Пороговая бинаризация

Пороговая (глобальная) бинаризация – это метод обработки изображений, который позволяет преобразовать цветное изображение или изображение в оттенках серого в черно-белое. Белый цвет соответствует пикселям, яркость которых выше определенного порога, а черный цвет – пикселям, яркость которых ниже этого порога. Процесс пороговой бинаризации состоит из следующих шагов.

1. Конвертация изображения в оттенки серого, если оно не было в таком формате.
2. Определение порогового значения, по которому будет происходить бинаризация.
3. Перебор всех пикселей изображения и сравнение их яркости с пороговым значением.
4. Если яркость пикселя выше порогового значения, то ему присваивается белый цвет, а если ниже – черный цвет.

Популярным методом глобальной бинаризации изображений является метод Оцу. Данный метод фактически представляет собой алгоритм вычисления порогового значения.

Пороговая бинаризация часто используется в обработке изображений для решения задачи сегментации. Например, можно использовать пороговую бинаризацию для выделения текста на странице или для выделения объектов с одинаковым цветом фона на изображении.

Однако следует учитывать, что пороговая бинаризация может не дать хорошего результата, когда на изображении есть области с разной освещенностью. В этом случае пороговые значения для различных областей изображения будут отличаться, и тогда более эффективным методом может быть использование адаптивной бинаризации, который позволяет определять пороговое значение для каждой области изображения отдельно.

Для применения глобальной бинаризации и автоматического выбора порога методом Оцу можно воспользоваться функцией `cv2.threshold` из библиотеки OpenCV. Первый аргумент – это исходное изображение, которое должно быть

изображением в градациях серого. Второй аргумент – это пороговое значение, которое используется для классификации значений пикселей. Третий аргумент – это максимальное значение, которое присваивается значениям пикселей, превышающим пороговое значение. OpenCV предоставляет различные типы пороговых значений, которые задаются четвертым параметром функции. Метод возвращает два результата. Первый – это использованный порог, а второй результат – изображение с пороговым значением.

Адаптивная бинаризация

Адаптивная бинаризация – это метод обработки изображений, который позволяет определить пороговое значение для каждой области изображения отдельно, в зависимости от особенностей освещения и контрастности в этой области. Это позволяет получить более точный результат по сравнению с пороговой бинаризацией, где одно и то же пороговое значение применяется для всего изображения.

Процесс адаптивной бинаризации состоит из следующих шагов.

1. Разбиение изображения на небольшие неперекрывающиеся блоки.
2. Расчет порогового значения для каждого блока. Обычно для этого используются методы вычисления статистических параметров яркости пикселей в блоке, таких как среднее значение или медиана.
3. Рассмотрение каждого пикселя изображения и сравнение его яркости с пороговым значением блока, к которому он принадлежит.
4. Если яркость пикселя выше порогового значения блока, то ему присваивается белый цвет, а если ниже – черный цвет.

Одним из главных преимуществ адаптивной бинаризации является возможность учитывать неоднородность освещения на изображении, что обычно является проблемой для пороговой бинаризации.

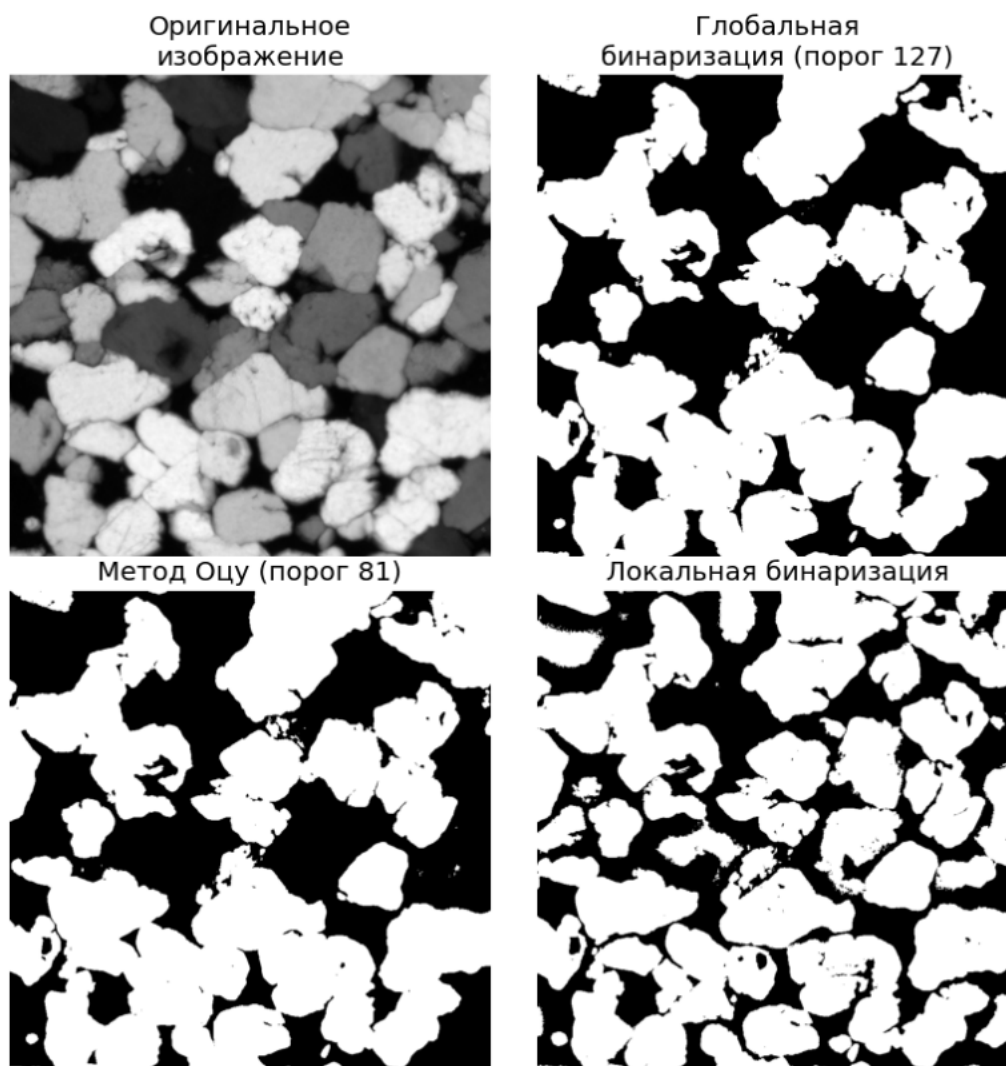


Рис. 1. Пример работы методов бинаризации

Для выполнения адаптивной бинаризации используется метод `cv2.adaptiveThreshold`. Помимо переменных, используемых в `cv2.threshold`, этот метод принимает три входных параметра:

- `cv2.ADAPTIVE_THRESH_MEAN_C` – пороговое значение, равное разности среднего значения площадей окрестностей (в пикселях) и константы `C`;
- `blockSize` – размер рассматриваемой области (в пикселях);
- `C` – величина, которая вычитается из средней или взвешенной суммы пикселей области.

На рисунке 1 приведены примеры изображений, обработанных различными методами бинаризации.

Контрольные вопросы

1. Чем отличается глобальная бинаризация от локальной?
2. В чем преимущество использования метода Оцу?
3. На каких типах изображений имеет преимущество адаптивная бинаризация?

Лабораторная работа

1. Напишите программу для бинаризации изображения, используя функции `cv2.threshold` и `cv2.adaptiveThreshold`. Сравните визуально полученные результаты (библиотека `matplotlib`) и обоснуйте выводы о методах бинаризации.
2. Напишите программу, которая будет выполнять глобальную бинаризацию изображения без использования метода `cv2.threshold`. Программа должна принимать на вход путь к файлу-изображению и пороговое значение, а сохранять бинаризованное изображение в другом файле.
3. Создайте простое графическое приложение с использованием библиотеки Tkinter, которое позволит пользователю выбирать изображение, метод бинаризации и соответствующие параметры, а затем выводить результат на экран.

ГЛАВА 2

МЕТОДЫ НА ОСНОВЕ РЕГИОНОВ

Метод сегментации Region Growing

Метод сегментации Region Growing основан на идее объединения пикселей в однородные области, содержащие схожие элементы (пиксели). Алгоритм начинается с выбора начального пикселя (зерна) в изображении и пошагово добавляет к нему соседние пиксели, которые подходят по какому-то критерию сходства. Таким образом, образуется растущая область (регион), состоящая из пикселей «схожего содержания».

Метод содержит следующие этапы.

1. Выбор начального пикселя: на первом шаге выбирается один пиксель как начальный пиксель для региона. Этот пиксель будет служить отправной точкой для роста региона.
2. Критерий сходства: выбирается критерий, который определяет, насколько похожи пиксели на уже растущий регион. Например, на основе заданных пороговых значений для яркости пикселя.
3. Процесс (этапы 1-2) продолжается до тех пор, пока регион не перестанет расширяться, то есть, когда все добавляемые пиксели уже включены в регион, или достигнут заданный размер региона.

Хотя метод Region Growing может дать хорошие результаты на простых изображениях с однородными регионами, он может столкнуться с проблемами на более сложных изображениях, где есть много перекрывающихся регионов.

На рисунке 2 приведен пример выделения одного объекта методом Region Growing. В качестве отправной точки алгоритма был выбран центральный пиксель. Критерий сходства: пороговое значение, равное 80 (значение выбрано экспериментально).

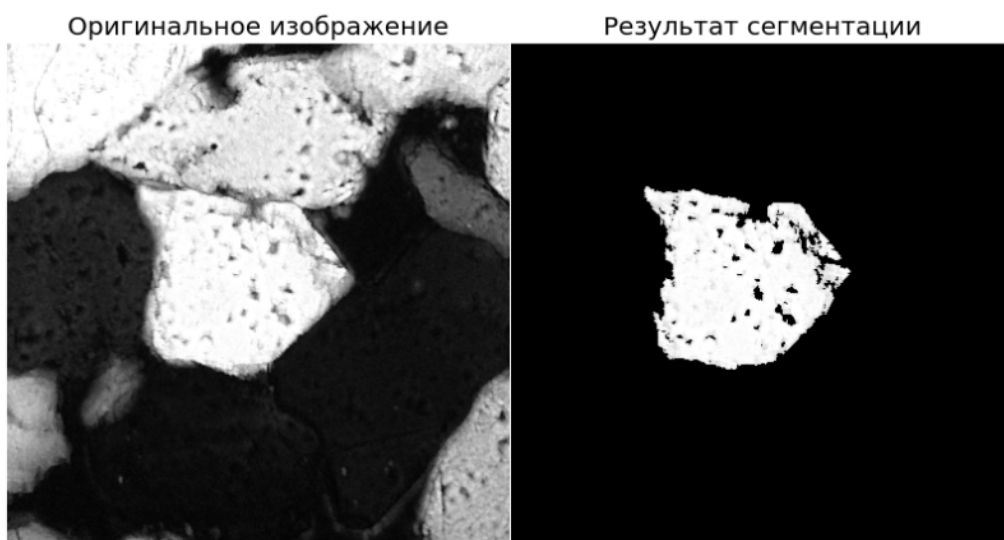


Рис. 2. Пример выделения одного региона с помощью Region Growing

Метод сегментации изображений Split-and-Merge

Метод сегментации изображений Split-and-Merge разделяет изображения на мелкие сегменты, а затем объединяет их в более крупные сегменты, основываясь на определенных критериях.

Основная идея метода Split-and-Merge заключается в следующем.

1. *Разделение (Split)*. На первом этапе изображение разделяется на четыре равные части (блоки). Далее в каждом блоке сравниваются значения интенсивности центрального пикселя со средним значением интенсивности всего блока. Если сравниваемые значения интенсивностей сильно отличаются, то блок делится на более мелкие части. Этот процесс повторяется для каждого созданного подсегмента до тех пор, пока не будут получены блоки, в которых все пиксели имеют близкую интенсивность. Отметим, что для определения схожести интенсивностей пикселей, можно, например, вычислить разницу между наибольшим и наименьшим значением интенсивности пикселей в блоке.
2. *Объединение (Merge)* На втором этапе объединяются соприкасающиеся блоки схожих интенсивностей, и окончательно получается сегментированное изображение. Объединение происходит, если два или более связанных сегментов удовлетворяют определенному критерию, например, если их разница в интенсивности пикселей достаточно мала.

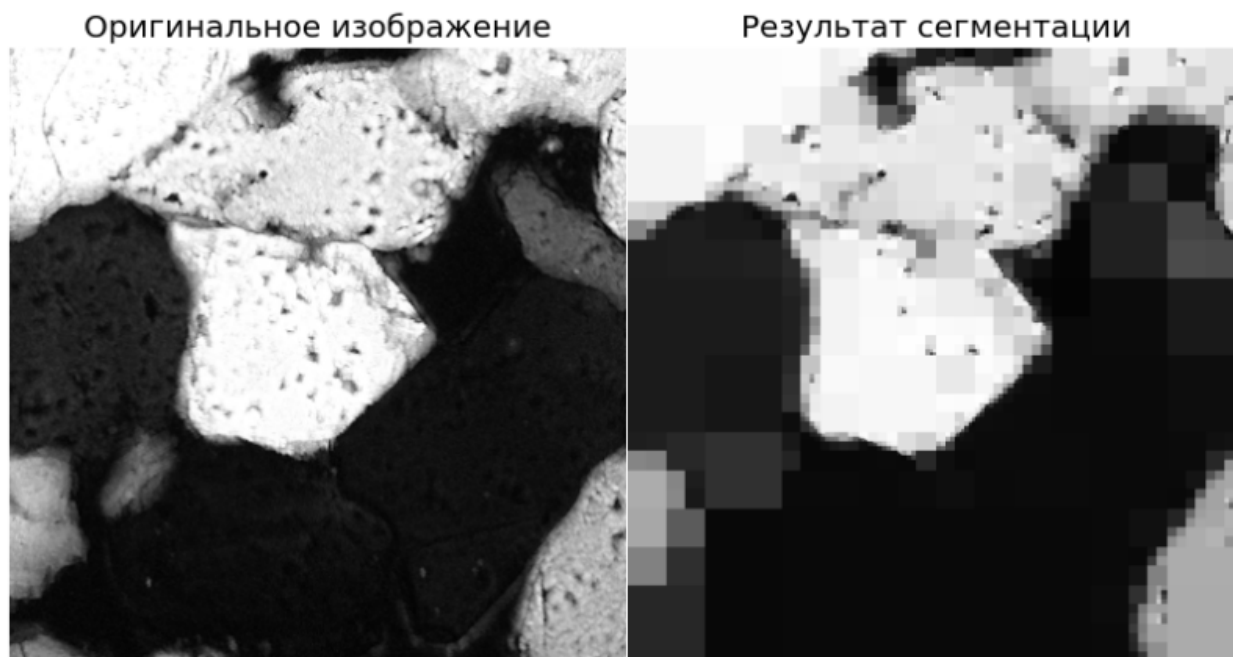


Рис. 3. Пример сегментации изображения методом Split-and-Merge

Сегментация методом водораздела

Сегментация водораздела (англ. Watershed Segmentation) – это один из самых популярных алгоритмов сегментации изображений, обычно используемый, когда нужно решить одну из самых сложных операций в обработке изображений – разделить похожие соприкасающиеся друг с другом объекты на изображении. Объясним подробно суть алгоритма. Представьте изображение в градациях серого как топографическую карту, и на этом изображении значения пикселей с высокой интенсивностью представляют собой пики (белые области), тогда как значения с низкой интенсивностью представляют долины – локальные минимумы (черные области).

Представьте теперь, что начинаем заполнять всю долину водой, и через некоторое время вода, поступающая из разных долин, начинает сливаться. Чтобы этого избежать, нужно построить барьеры в местах слияния воды, поэтому барьеры называются линиями бассейна и используются для определения границ сегмента. Когда вода наполнит всю территорию, вплоть до самого высокого пика, то заливку водой остановим. В конце процесса будут видны только линии водораздела (построенные барьеры), и это будет окончательным результатом сегментации. Таким образом, целью алгоритма водораздела является построение линий водораздела, которые и будут сегментировать изображение.

Алгоритм водораздела содержит следующие этапы.

1. Получение бинарного изображения.
2. Вычисление преобразования расстояния.
3. Нахождение локальных точек максимума.
4. Обозначение меток.

Рассмотрим каждый этап, приводя код на Python, реализующий его. Сначала импортируем необходимые библиотеки.

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from scipy import ndimage as ndi
from skimage.segmentation import watershed
from skimage.feature import peak_local_max
from skimage import morphology
from skimage.measure import label
```

Для получения бинарного изображения, которое будет «удобно» сегментировать, необходимо провести предобработку исходного изображения. С этой целью реализуем необходимые для предобработки изображения функции. Ниже приведены методы для выполнения морфологических операций: эрозии (уменьшения изображения) и дилатации (увеличения изображения).

```
def erode(img, k_s=3, it=1):
    kernel = np.ones((k_s, k_s), 'uint8')
    erode_img = cv2.erode(img, kernel, iterations=it)
    return erode_img
def dilate(img, k_s=3, it=1):
    kernel = np.ones((k_s, k_s), 'uint8')
    dilate_img = cv2.dilate(img, kernel, iterations=it)
    return dilate_img
```

Эрозия – это математическая операция, которая используется для сужения или уменьшения объекта на изображении. Она основана на логической операции

AND между объектом и структурным элементом. Структурный элемент представляет собой небольшую геометрическую фигуру, такую как точка или квадрат, которая проходит по изображению. Если все пиксели объекта соответствуют пикселям структурного элемента, то эти пиксели сохраняются, в противном случае они удаляются. Таким образом, результатом эрозии является сужение объекта на изображении.

Дилатация – это математическая операция, которая используется для расширения или увеличения объекта на изображении. Она основана на логической операции OR между объектом и структурным элементом. Структурный элемент также используется для изменения изображения, и если хотя бы один пиксель объекта соответствует пикселю структурного элемента, то все пиксели структурного элемента считаются частью объекта. Таким образом, результатом дилатации является расширение объекта на изображении.

Эрозия и дилатация широко используются в области обработки изображений для улучшения качества изображения или выделения интересующих областей. Например, эрозия может использоваться для удаления шума или неправильных пикселей, а дилатация может использоваться для заполнения пробелов или увеличения размера объектов.

Эти процессы часто комбинируются вместе для достижения желаемого результата. Например, применение эрозии перед дилатацией может помочь удалить мелкие дефекты и затем расширить оставшиеся области. Этот процесс называется открытием. С другой стороны, применение дилатации перед эрозией может помочь заполнить пробелы и затем сузить области. Этот процесс называется закрытием.

Эрозия и дилатация являются основными операциями в математической морфологии, что представляет собой технику обработки и анализа изображений. Они играют важную роль во многих приложениях, таких как распознавание образов, сегментация изображений, а также в медицинском и биологическом анализе данных.

Функция для удаления небольших объектов на бинарном изображении. Используется метод `morphology.remove_small_objects`, который на вход принимает изображение с метками областей от 1 до n (количество областей на изображении).

```
def del_small_areas(thresh, area_black = 100,
area_white=100):
    result = morphology.remove_small_objects(label
(thresh), area_white,)
    result[result>0]=255
    result = morphology.remove_small_objects(label
(255-result), area_black,)
    result[result>0]=255
    result = 255 - result
    return result
```

Проводим локальную бинаризацию изображения с последующей обработкой (эрозия, дилатация и удаление мелких объектов).

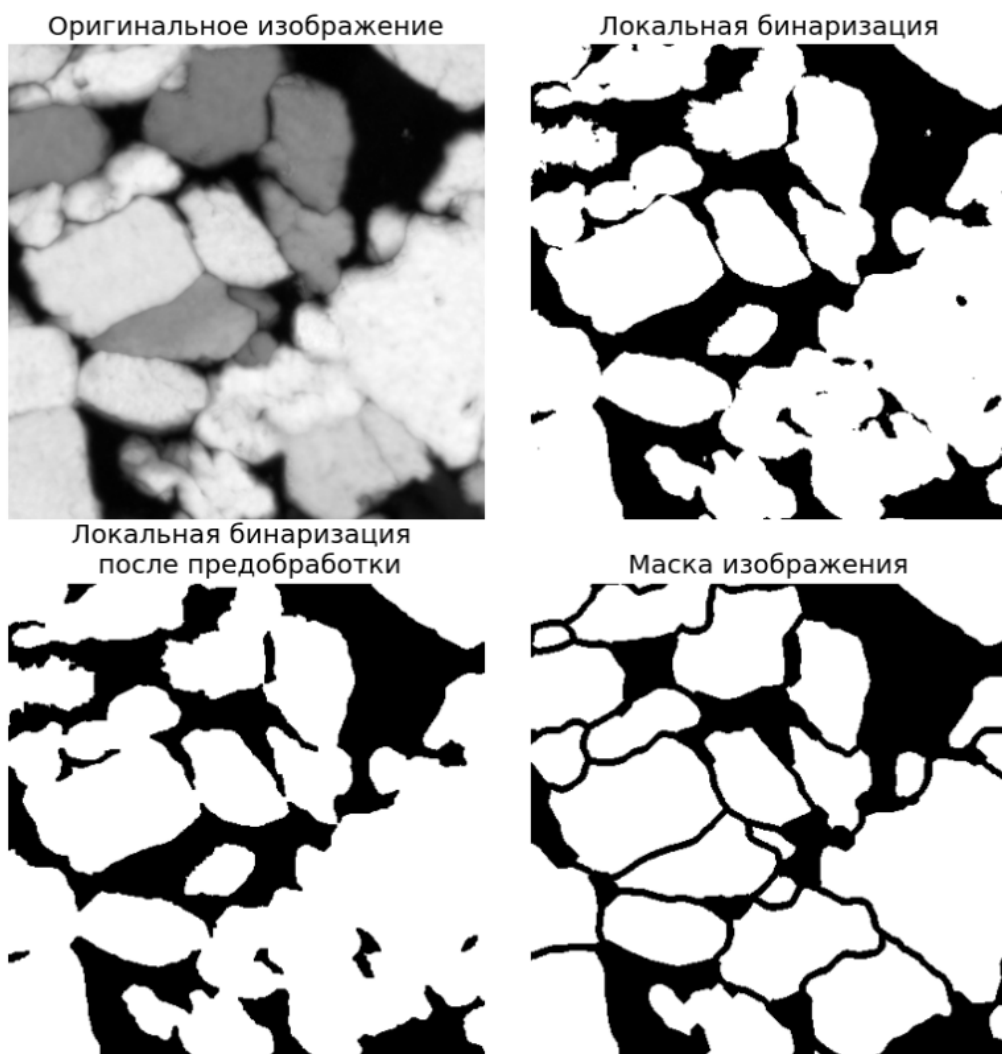


Рис. 4. Результат локальной бинаризации

```

img = cv2.imread('data/image.png',
cv2.IMREAD_GRAYSCALE)
mask_image =
cv2.imread('data/mask.png', cv2.IMREAD_GRAYSCALE)
thresh_local_1 = cv2.adaptiveThreshold(img, 255,
cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY_INV,
101, 10)
thresh_local =
del_small_areas(erode(dilate(thresh_local_1)))

```

Теперь нужно попытаться разделить соприкасающиеся объекты и создать границу между ними. Идея состоит в том, чтобы создать границу как можно дальше от центров перекрывающихся объектов. Затем используем алгоритм, называемый преобразованием расстояния. Этот алгоритм получает двоичные изображения в качестве входных данных. На выходе алгоритм получает изображения, содержащие информацию (в виде интенсивности пикселей) о расстоянии от текущего пикселя до ближайшего пикселя с нулевой интенсивностью (фоновый пиксель). Для этого можно использовать функцию `Distance_transform_edt()` из библиотеки `scipy`. Эта функция преобразует исходное изображение в изображение «преобразование расстояния», используя классическую евклидову метрику:

```
distance = ndi.distance_transform_edt(thresh_local).
```

Приведем на рисунке 5 исходное изображение, бинарное изображение и преобразование расстояния.

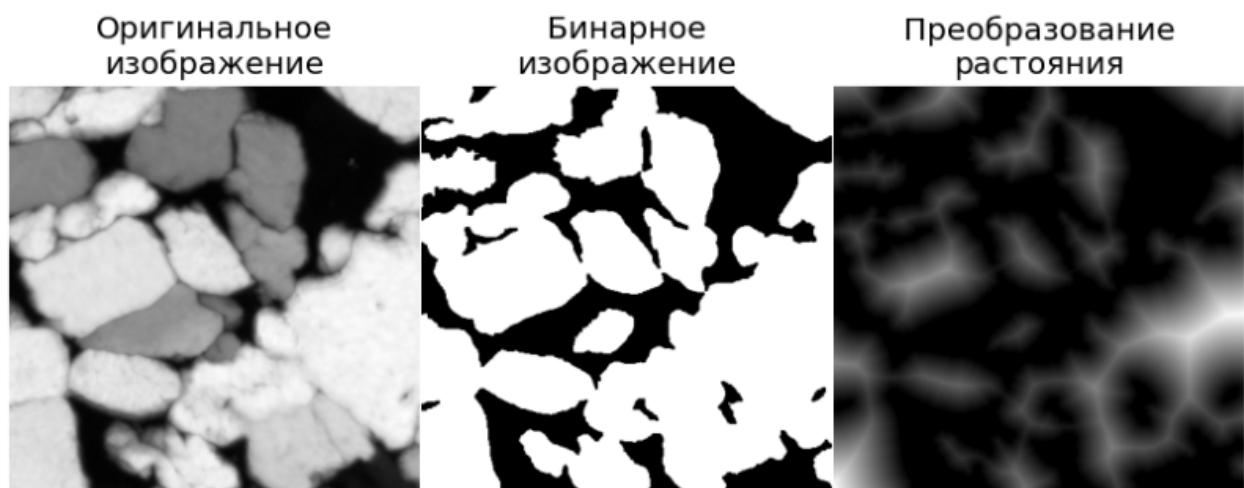


Рис. 5. Преобразование расстояния

Обратите внимание, что в преобразовании расстояния можно видеть белые пиксели, эти пиксели имеют большие значения преобразования расстояния (другими словами, большие расстояния до фоновых пикселей). Теперь нужно найти координаты пиков (локальных максимумов) белых областей на изображении. Для этого используем функцию `peak_local_max()` из библиотеки `Scikit-image`. Применим эту функцию к построенному изображению `Distance_transform` и получим маркеры, которые будут использованы в функции водосбора.

```
local_max = peak_local_max(distance, min_distance=50,  
footprint=np.ones((3, 3)), labels=thresh_local)
```

Следующим шагом является разметка полученных маркеров для функции водораздела. Для этого воспользуемся функцией `ndi.label()` из библиотеки `SciPy`. Эта функция состоит из одного входного параметра, который представляет собой объект, подобный массиву, который нужно пометить. Любые ненулевые значения этого параметра считаются признаками, а нулевые значения считаются фоном. В нашей программе будем использовать координаты рассчитанного локального максимума. Переведем эти координаты в двумерный массив и функцией `ndi.label()` случайным образом пометим все локальные максимумы разными положительными значениями, начиная с 1. Таким образом, если получим `N` объектов на изображении, каждый из которых будет помечен значением от 1 до `N`.

```
mask = np.zeros(distance.shape, dtype=bool)  
mask[tuple(local_max.T)] = True  
markers, _ = ndi.label(mask)
```

На последнем шаге применим функцию `skimage.segmentation.watershed()` из библиотеки `Scikit-image`. В качестве параметров нужно передать инвертированное изображение преобразования расстояния и маркеры, которые вычислили в строках кода, представленных немного выше. Поскольку алгоритм водораздела предполагает, что маркеры представляют собой локальные минимумы, нужно инвертировать

изображение преобразования расстояния. Таким образом, светлые пиксели будут представлять большие возвышения, а темные пиксели будут представлять низкие возвышения для преобразования водораздела:

```
labels = watershed(-distance, markers,  
mask=thresh_local).
```

Далее при помощи функций `cv2.findContours` и `cv2.drawContours` найдем и выделим границы полученных областей на исходном изображении.

```
image_countours =  
cv2.cvtColor(img.copy(), cv2.COLOR_GRAY2BGR)  
for label in np.unique(labels):  
    if label == 0:  
        continue  
    mask = np.zeros(thresh_local.shape, dtype="uint8")  
    mask[labels == label] = 255  
    cnts = cv2.findContours(mask.copy(),  
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)  
    cnts = cnts[0] if len(cnts) == 2 else cnts[1]  
    cv2.drawContours(image_countours, cnts, -1,  
(0, 255, 0), 1)
```

В следующих строках происходит визуализация полученных результатов сегментации с помощью библиотеки `matplotlib`:

```
fig, axes = plt.subplots(ncols=4, figsize=(10, 5),  
sharex=True, sharey=True)  
ax = axes.ravel()  
  
ax[0].imshow(gray, cmap=plt.cm.gray)  
ax[0].set_title('Оригинальное\nизображение')  
  
ax[1].imshow(labels, cmap=plt.cm.nipy_spectral)  
ax[1].set_title('Сегментация\nводораздела')
```

```

ax[2].imshow(image_countours,
cmap=plt.cm.nipy_spectral)
ax[2].set_title('Найденные\nграницы')

ax[3].imshow(mask_image, cmap=plt.cm.gray)
ax[3].set_title('Исходная маска\nсегментации')

for a in ax:
    a.set_axis_off()

fig.tight_layout()
plt.show()

```

Приведем исходное изображение, изображение водораздела, найденные границы на изображении и исходную маску сегментации на рисунке 6.

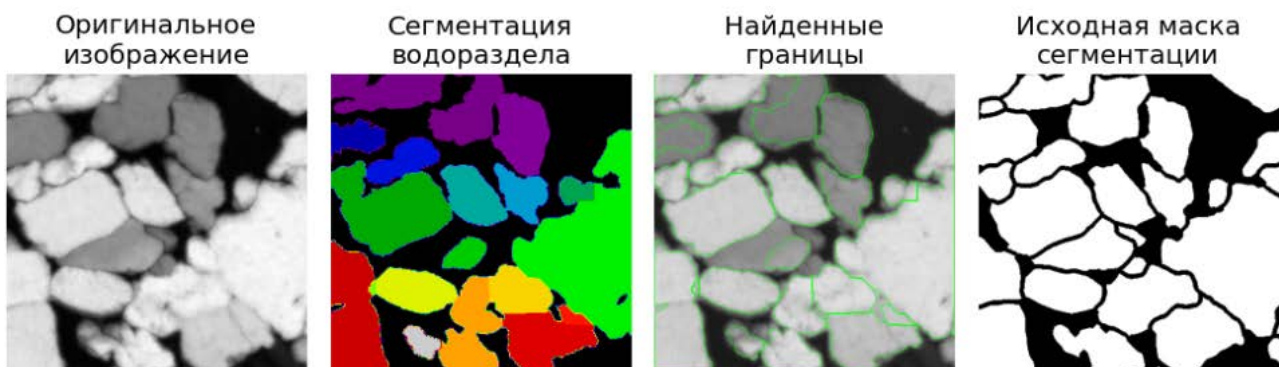


Рис. 6. Результат сегментации методом водораздела

Контрольные вопросы

1. Чем метод Region Growing отличается от метода Split-and-Merge?
2. Какой принцип лежит в основе метода Watershed Segmentation?
3. Какие этапы включает алгоритм водораздела?
4. Какие библиотеки используются в предоставленном коде для обработки изображений?

Лабораторная работа

1. Реализуйте функцию, которая будет применять метод Region Growing к заданному изображению. Начните с выбора начального пикселя и итеративного добавления пикселей на основе заданного порога.

2. Изучите метод Split-and-Merge и напишите функцию, которая будет разделять и объединять сегменты на основе заданных критериев. Примените этот метод к изображению и сравните результаты с другими методами.

3. Примените метод Watershed Segmentation к изображениям. Используйте функции из предоставленного кода для обработки и анализа. Исследуйте влияние изменения параметров и методов предобработки (например, размера окна адаптивной бинаризации, пороговых значений, `min_distance` в методе `peak_local_max` и т.д.) на результаты сегментации. Сделайте выводы о том, какие параметры оказывают наибольшее влияние на результаты.

ГЛАВА 3

МЕТОДЫ СЕГМЕНТАЦИИ ИЗОБРАЖЕНИЙ НА ОСНОВЕ ГРАНИЦ

Оператор Собеля

Оператор Собеля – это классический оператор обработки изображений, используемый для выделения границ. Он был предложен Ирвином Собелем и Гарретом Коэном в 1968 году, и с тех пор стал одним из наиболее распространенных операторов для обработки изображений.

Оператор Собеля использует ядро (также называемое маской) размером 3x3, которое применяется к каждому пикселю изображения. Ядро содержит числа, которые определяют вклад каждого пикселя в окрестности в конечный результат.

-1	0	+1
-2	0	+2
-1	0	+1

G_x

+1	+2	+1
0	0	0
-1	-2	-1

G_y

Рис. 7. Ядра для обнаружения границ: G_x и G_y

Оператор Собеля использует два ядра – одно для обнаружения границ по горизонтали, а другое для обнаружения границ по вертикали. Ядро G_x используется для обнаружения границ по горизонтали, а ядро G_y используется для обнаружения границ по вертикали. Для каждого пикселя изображения вычисляются свертки¹ G_x и G_y с элементами матрицы изображения такого же размера с центром в данном пикселе.

Затем для каждого пикселя вычисляется значение G как квадратный корень суммы квадратов значений G_x и G_y для этого пикселя:

$$G = \sqrt{G_x^2 + G_y^2}$$

¹ Свертка двух матриц – перемножение элементов с одинаковыми индексами двух матриц между собой с последующим суммированием полученных произведений.

После вычисления значения G для каждого пикселя изображения можно задать пороговое значение, чтобы определить, какие пиксели будут считаться границами. Пиксели со значением, превышающим пороговое значение, будут считаться границами, а остальные – неграничными элементами.

```
import cv2
# загружаем изображение
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# применяем оператор Собеля для обнаружения границ
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
# объединяем результаты
sobel = cv2.sqrt(cv2.addWeighted(cv2.pow(sobelx, 2.0),
1.0, cv2.pow(sobely, 2.0), 1.0, 0.0))
# применяем пороговую бинаризацию
sobel_thresh = cv2.threshold(sobel, 100, 255,
cv2.THRESH_BINARY)[1]
```

В приведенном примере загружаем изображение с помощью функции `cv2.imread()`, применяем оператор Собеля для обнаружения границ с помощью функций `cv2.Sobel()` и объединяем результаты с помощью функций `cv2.sqrt()` и `cv2.addWeighted()`. Затем применяем пороговую бинаризацию для выделения границ.

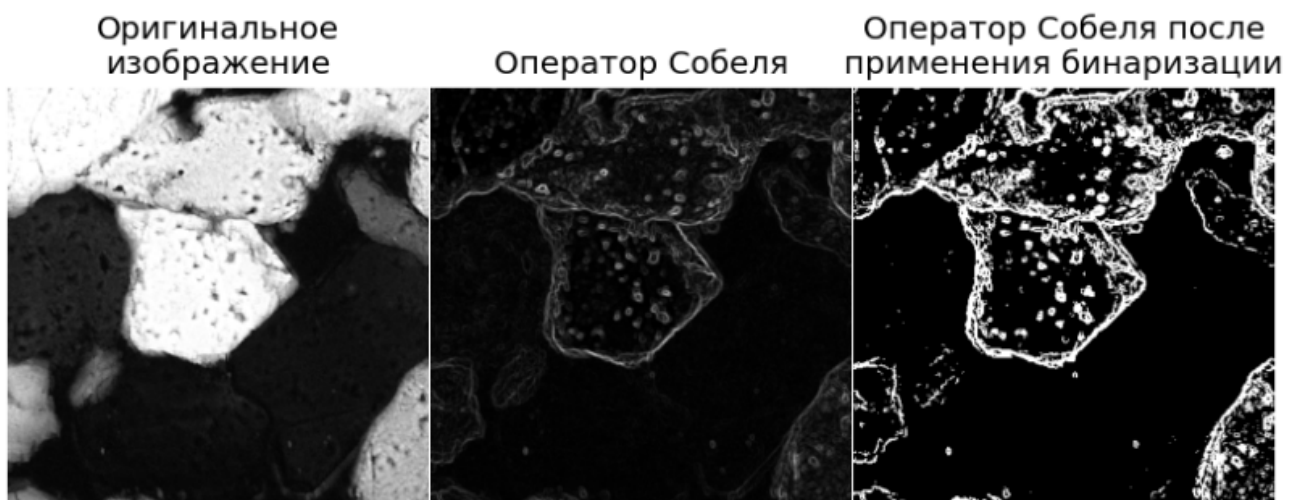


Рис. 8. Результат работы оператора Собеля

Оператор Собеля является простым и эффективным методом обработки изображений для выделения границ. Он может использоваться как самостоятельный метод, так и в комбинации с другими методами сегментации изображений.

Оператор Кэнни

Оператор Кэнни – это алгоритм обнаружения границ на изображении, который был разработан Джоном Кэнни в 1986 году. Этот оператор границ является более сложным и точным, чем оператор Собеля. Он использует несколько этапов обработки изображения, включая размытие, поиск градиентов, подавление не-максимумов и пороговую фильтрацию.

1. Сглаживание изображения: на первом этапе изображение проходит через фильтр Гаусса, чтобы уменьшить шум и сгладить изображение.

2. Вычисление градиента: на следующем этапе вычисляются градиенты изображения в направлении x и y . Для этого обычно используется оператор Собеля.

3. Подавление не-максимумов: затем выполняется подавление не-максимумов, которое помогает удалить все точки, которые не являются максимальными в своем окружении. Это делается для того, чтобы уменьшить количество границ и сделать их более четкими.

4. Двойная пороговая фильтрация: после этого применяется двойная пороговая фильтрация. Она позволяет выделить границы на изображении, удаляя пиксели, которые не превышают заданный нижний порог, и соединяя только те пиксели, которые превышают заданный верхний порог.

5. Определение связных компонентов: наконец, определяются связные компоненты на изображении, чтобы выделить сегменты.

Ниже приведен пример кода на Python, который демонстрирует использование оператора Кэнни для сегментации изображений.

```
import cv2
# загрузка изображения и преобразование в оттенки
серого
img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# применение оператора Кэнни
edges = cv2.Canny(gray, 100, 200)
# поиск контуров
contours, _ = cv2.findContours(edges, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
# отрисовка контуров на изображении
cv2.drawContours(img, contours, -1, (0, 255, 0), 1)
# отображение результата
```

В приведенном примере загружаем изображение и преобразуем его в оттенки серого с помощью функции `cv2.cvtColor()`. Затем применяем оператор Кэнни с помощью функции `cv2.Canny()` и находим контуры с помощью функции `cv2.findContours()`. После этого отрисовываем контуры на исходном изображении с помощью функции `cv2.drawContours()`.

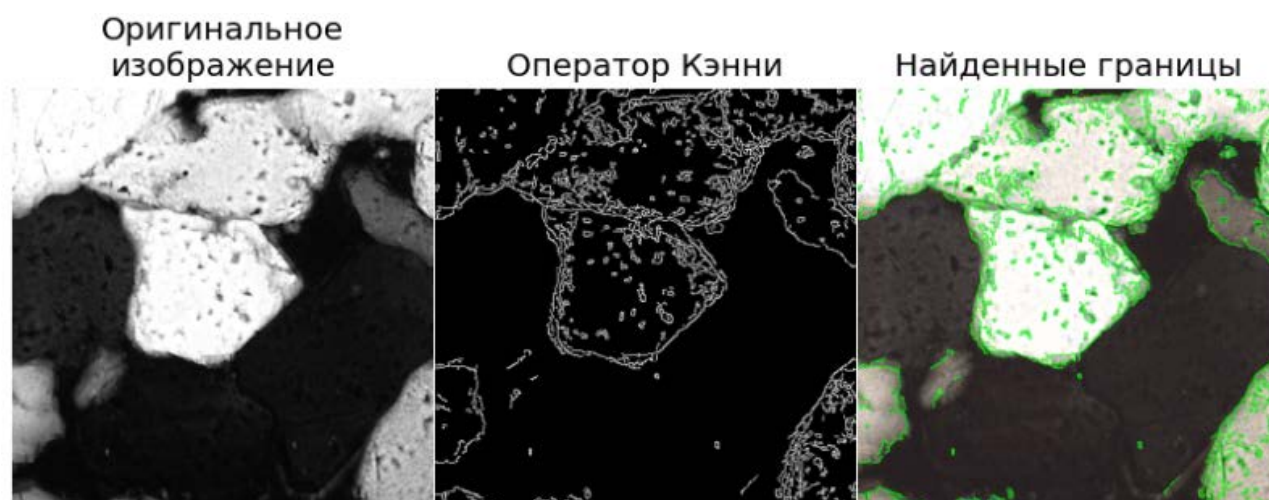


Рис. 9. Результат работы оператора Кэнни

Обратите внимание, что оператор Кэнни требует настройки двух пороговых значений: нижнего и верхнего. Эти значения могут быть различными в зависимости от конкретной задачи и изображения. Нижний порог определяет минимальное значение градиента, необходимое для того, чтобы пиксель был рассмотрен как часть границы, а верхний порог определяет минимальное значение градиента, необходимое для того, чтобы пиксель был рассмотрен как часть «сильной» границы.

Одним из главных преимуществ оператора Кэнни является то, что он способен находить границы объектов с высокой точностью и четкостью. Однако его недостатком является то, что он может быть чувствителен к шуму на изображении. Поэтому перед применением оператора Кэнни рекомендуется выполнить сглаживание изображения для удаления зашумленных пикселей.

Контрольные вопросы

1. Какие два ядра используются в операторе Собеля для обнаружения границ?
2. Каким образом объединяются результаты применения ядер G_x и G_y в операторе Собеля?
3. На какие этапы разделяется алгоритм Кэнни для обнаружения границ?

Лабораторная работа

1. Напишите код с использованием оператора Собеля для выделения границ на изображении. Загрузите изображение и примените оператор Собеля по горизонтали и по вертикали.
2. Возьмите любое изображение и напишите программу для обнаружения границ с помощью оператора Кэнни. Исследуйте влияние изменения пороговых значений оператора Кэнни на результат сегментации.
3. Попробуйте применить оператор Собеля и оператор Кэнни к разным типам изображений (например, фотографии пейзажей и изображения с объектами). Сравните результаты и опишите, какой оператор дал более четкие и точные границы для каждого типа изображения.

ГЛАВА 4

МЕТОДЫ НА ОСНОВЕ КЛАСТЕРИЗАЦИИ

K-Means Clustering

K-Means Clustering – это алгоритм кластеризации, который используется для разбиения изображения на сегменты, основываясь на их сходстве в пространстве цветовых характеристик. Он основан на группировке пикселей на основе их цвета в заданное количество кластеров.

Основная идея метода K-Mean Cluster заключается в следующем:

- 1) выбрать количество кластеров, на которые будет разделено изображение;
- 2) выбрать случайным образом центроиды (средние значения) для каждого кластера;
- 3) оценить расстояние между каждым пикселем и каждым центроидом;
- 4) назначить каждый пиксель к ближайшему кластеру на основе расстояния до его центроида;
- 5) обновить центроиды кластеров на основе новых средних значений цвета для пикселей в каждом кластере;
- 6) повторять шаги 3–5 до тех пор, пока центроиды не перестанут изменяться или до достижения максимального количества итераций.

После выполнения алгоритма каждый пиксель будет назначен к одному из кластеров, при этом каждый кластер будет представлять собой группу пикселей с близкими значениями цветовых характеристик. Эти кластеры могут быть использованы для разделения изображения на сегменты. В конечном результате выводятся исходное изображение и сегментированное изображение с помощью метода K-Means Clustering (рисунок 10).

```
import numpy as np
import cv2

def kmeans_segmentation(image, k):
    # Преобразование изображения в массив пикселей
    pixels = image.reshape(-1, 3).astype(np.float32)

    # Выполнение K-Means сегментации
```

```

        criteria = (cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
        _, labels, centers = cv2.kmeans(pixels, k, None,
criteria, 10, cv2.KMEANS_RANDOM_CENTERS)

        # Преобразование меток обратно в размеры
изображения
        segmented_image =
centers[labels.flatten()].reshape(image.shape)

    return segmented_image

# Загрузка изображения
image = cv2.imread('img.png')

# Применение K-Means сегментации с 2 кластерами
k = 2
segmented_image = kmeans_segmentation(image,
k).astype('uint8')

```

В данной программе использована функция `cv2.kmeans` для выполнения сегментации методом K-Means. Она принимает на вход несколько параметров:

- `pixels`: массив пикселей изображения;
- `k`: количество сегментов или кластеров;
- `None`: параметр для передачи начальных центров кластеров, в данном случае они выбираются случайным образом;
- `criteria (type, max_iter, epsilon)`: критерии остановки для алгоритма K-Means, здесь используется комбинация критериев `cv2.TERM_CRITERIA_EPS` и `cv2.TERM_CRITERIA_MAX_ITER`, первый критерий `cv2.TERM_CRITERIA_EPS` означает, что алгоритм завершается, если отклонение между двумя последовательными итерациями становится меньше определенного значения `eps`, второй критерий `cv2.TERM_CRITERIA_MAX_ITER` ограничивает максимальное количество итераций алгоритма;
- `10`: количество запусков алгоритма K-Means с разными начальными центрами;

– `cv2.KMEANS_RANDOM_CENTERS`: флаг для выбора начальных центров кластеров случайным образом;

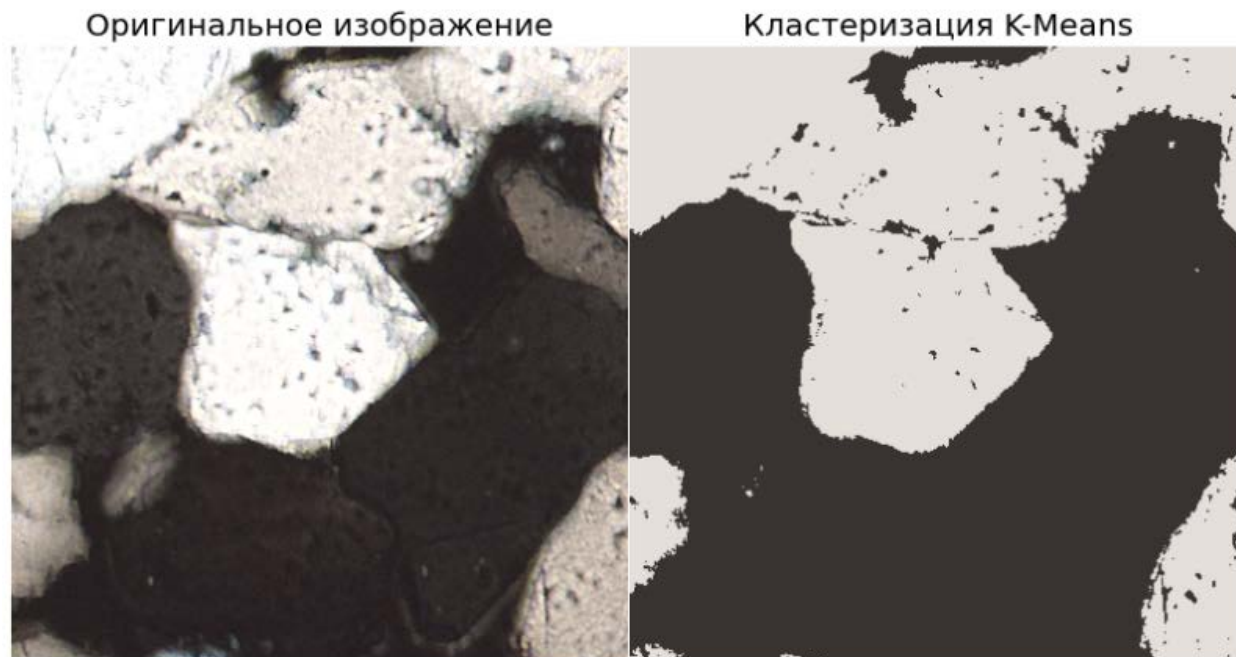


Рис. 10. Результат сегментации K-Means с двумя кластерами

Функция `cv2.kmeans` возвращает

- `labels`: массив меток, где каждый пиксель помечен соответствующим кластером;
- `centers`: массив центроидов каждого кластера.

Mean Shift

Mean Shift – это алгоритм кластеризации, который используется для разбиения изображения на сегменты, основываясь на их сходстве в пространстве цветовых характеристик. Он основан на группировке пикселей по цвету в заданное количество кластеров. Этот алгоритм является одним из алгоритмов машинного обучения без учителя, который позволяет выявить кластеры (группы) данных в многомерном пространстве.

Принцип работы метода Mean Shift представлен ниже.

1. Инициализация центроидов: начнем с выбора начальных центроидов, которые будут использоваться как стартовые точки для процесса сдвига среднего значения.

2. **Вычисление плотности:** для каждой точки данных вычисляется «плотность», которая определяется на основе расстояния между этой точкой и остальными точками. Это может быть сделано через функцию-ядро (например, гауссово ядро).

3. **Сдвиг среднего значения:** для каждой точки данных вычисляется сдвиг среднего значения в направлении более плотной области данных. Этот сдвиг вычисляется путем взвешенной суммы векторов направлений от точки данных до других точек, пропорционально их плотности.

4. **Обновление центроидов:** новые центроиды вычисляются путем сдвига старых центроидов на величину, равную сдвигу среднего значения.

5. **Повторение процесса:** шаги 3 и 4 выполняются итерационно до тех пор, пока центроиды не перестанут смещаться или пока не будет достигнуто максимальное количество итераций.

6. **Присвоение кластеров:** каждая точка данных присваивается к ближайшему центроиду, формируя таким образом кластеры.

7. **Постпроцессинг:** иногда производится постпроцессинг, чтобы объединить слишком близкие кластеры или удалить выбросы.

В контексте сегментации изображений пиксели рассматриваются как данные (точки в многомерном пространстве), определяемые их цветом, текстурой и другими характеристиками. Путем применения метода Mean Shift к этим пикселям можно найти группы пикселей со схожими характеристиками, что позволяет выделить сегменты объектов и фоновых областей на изображении.

Преимущества метода Mean Shift:

1) **автоматическое определение количества кластеров:** Mean Shift не требует указания числа кластеров заранее, метод сам определяет количество кластеров;

2) **обработка различных размеров кластеров:** алгоритм способен обнаруживать кластеры различных размеров и форм;

3) **устойчивость к выбросам:** благодаря процессу сдвига среднего значения алгоритм менее подвержен влиянию выбросов.

Однако метод Mean Shift также имеет некоторые ограничения:

1) **вычислительная сложность:** алгоритм может быть вычислительно затратным, особенно для больших наборов данных;

2) **зависимость от параметров:** для определения параметров (например, размер окна для вычисления сдвига среднего значения) может быть необходимо провести некоторые эксперименты;

3) плотность данных: если данные имеют неравномерное распределение плотности, алгоритм может давать менее точные результаты.

Метод Mean Shift может быть эффективным инструментом для сегментации изображений, но его успешное применение требует тщательного подбора параметров и обработки результатов. Для реализации данного метода на языке Python можно воспользоваться функцией `MeanShift()` из библиотеки `sklearn`. На рисунке 11 приведен результат кластеризации.

```
import numpy as np
from sklearn.cluster import MeanShift,
estimate_bandwidth

image = cv2.imread('image.png')

# Преобразование изображения в массив пикселей
pixels = np.reshape(image, (-1, 3))

# Оценка параметра bandwidth
bandwidth = estimate_bandwidth(pixels, quantile=0.2,
n_samples=500)

# Создание объекта MeanShift с найденным параметром
bandwidth
ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)

# Обучение модели на данных
ms.fit(pixels)

# Получение меток кластеров
labels = ms.labels_

# Преобразование меток кластеров в 2D-матрицу
labels_2d = np.reshape(labels, image.shape[:2])
```

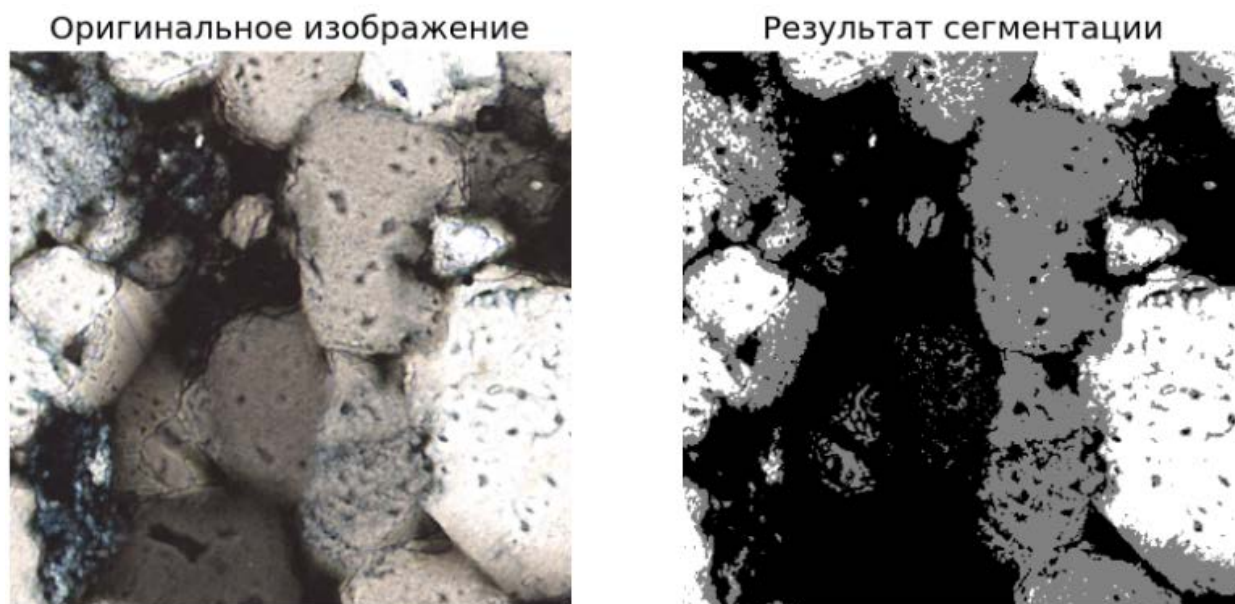


Рис. 11. Результат кластеризации пикселей с помощью алгоритма Mean Shift (было найдено три кластера)

Параметр `bandwidth` и `bin_seeding` – это два важных параметра метода кластеризации `MeanShift()`, которые влияют на его производительность и результаты. Оценка параметра `bandwidth`:

- параметр `bandwidth` определяет размер окрестности, используемой для оценки плотности точек в пространстве при кластеризации, он влияет на масштаб и гладкость сегментации;

- для получения оптимального значения `bandwidth` можно использовать функцию ``estimate_bandwidth`` из модуля `sklearn.cluster`, эта функция оценивает параметр `bandwidth`;

- в параметре `quantile` можно указать, какую долю точек использовать при оценке параметра `bandwidth`. Меняя это значение, можно получить различные уровни детализации сегментации.

Параметр `bin_seeding` влияет на скорость сегментации. Если `bin_seeding` установлен в `True`, то начальные точки берутся из бинарной сетки, что может помочь ускорить алгоритм, но может привести к потере точности в определении центроидов.

Выбор и настройка этих параметров должны осуществляться исходя из конкретной задачи и особенностей данных. Рекомендуется проводить эксперименты с разными значениями и наблюдать, как они влияют на качество результатов и производительность алгоритма.

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) – это алгоритм кластеризации, который может использоваться для сегментации изображений. Он основан на плотности точек (данных) в пространстве и может идентифицировать кластеры произвольной формы. Процесс сегментации изображений на основе DBSCAN состоит из следующих шагов, представленных ниже.

1. Подготовка данных. Изображение должно быть преобразовано в набор точек, таких как пиксели или векторы признаков, для использования алгоритмом DBSCAN.

2. Определение параметров DBSCAN. Алгоритм DBSCAN имеет два основных параметра:

- Эпсилон (ϵ) – максимальное расстояние между двумя точками, чтобы они считались соседними.

- Минимальное количество точек (minsamples) – минимальное количество точек в окрестности, чтобы точка считалась ядром (core point).

3. Выполнение DBSCAN. На этом шаге применяется алгоритм DBSCAN к данным. Он начинается с выбора случайной неисследованной точки данных и ищет всех ее соседей в пределах радиуса ϵ . Если количество соседей больше или равно minsamples , они считаются ядром и образуют кластер. Затем процесс повторяется с каждой точкой-ядром и ее соседями, пока не будут рассмотрены все точки.

4. Постобработка кластеров. После кластеризации каждая точка будет принадлежать одному из кластеров или считаться выбросом (outlier). Выбросы – это точки данных, которые не имеют достаточного количества соседей, чтобы стать ядром. В рамках сегментации изображений выбросы могут представлять фоновые или шумовые пиксели.

Преимущества DBSCAN в сегментации изображений включают способность кластеризовать данные произвольной формы и устойчивость к выбросам. Однако алгоритм является чувствительным к выбору параметров, таким как ϵ и minsamples , и может потребовать определенной настройки для каждого конкретного изображения или типа данных.

Ниже приведем код и результат кластеризации на рисунке 12.

```

import numpy as np
from sklearn.cluster import DBSCAN
import cv2

# Загрузка изображения
image = cv2.imread('image.png')
# Преобразование RGB изображения в одномерный массив
pixels = image.reshape(-1, 3)

# Создание объекта DBSCAN
dbscan = DBSCAN(eps=3, min_samples=10)

# Производим кластеризацию пикселей
labels = dbscan.fit_predict(pixels)

# Преобразуем метки в изображение
segmented_image = labels.reshape(image.shape[0],
image.shape[1])

```

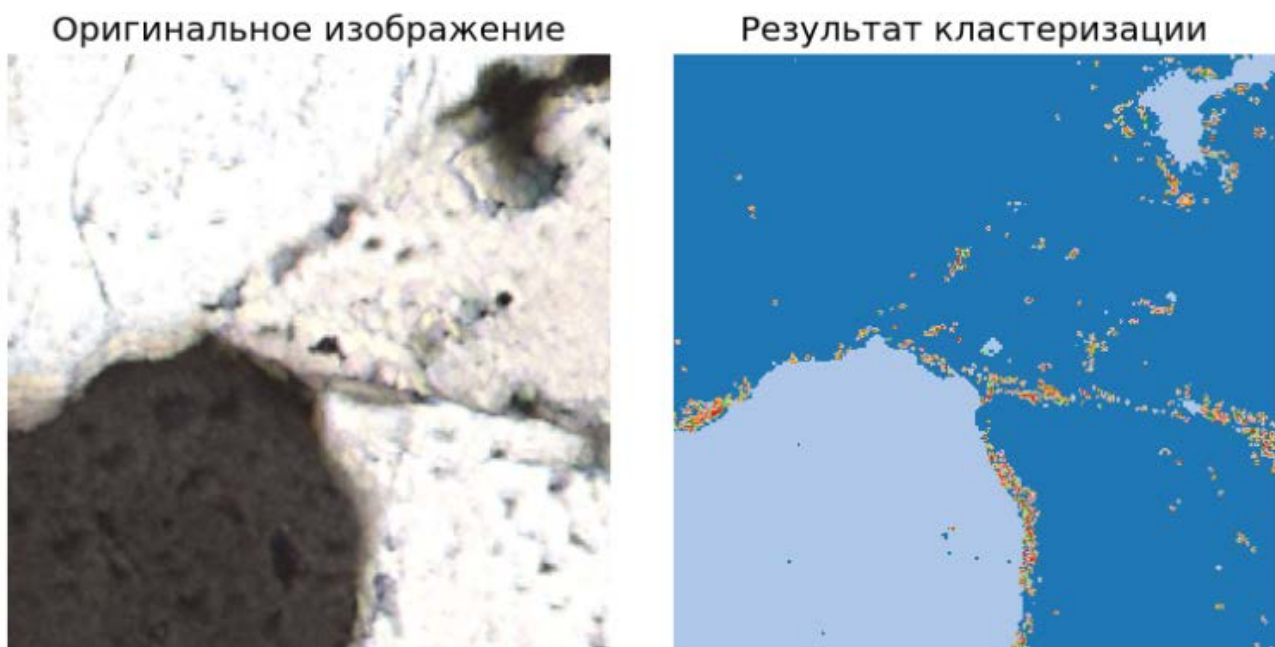


Рис. 12. Результат кластеризации методом DBSCAN

Контрольные вопросы

1. Что такое K-Means Clustering?
2. Какие основные шаги включает алгоритм K-Means Clustering?
3. Какие параметры передаются в функцию `cv2.kmeans()` в OpenCV?
4. Какие преимущества и ограничения имеет метод MeanShift?
5. Какие основные параметры влияют на работу метода DBSCAN?

Лабораторная работа

1. Расширьте функцию `kmeans_segmentation` из представленного кода, чтобы она принимала дополнительный параметр `max_iterations` и передавала его в функцию `kmeans`.
2. Примените `kmeans` к нескольким изображениям. Изменяя такие параметры, как количество кластеров, максимальное количество итераций, попробуйте получить оптимальный результат.
3. Попробуйте разные комбинации значений `eps` и `min_samples` на одном и том же изображении для алгоритма DBSCAN. Приведите визуальные результаты кластеризации для каждой комбинации и оцените, какие параметры лучше подходят для сегментации.
4. Выберите несколько изображений и выполните сравнительный анализ, оценив качество сегментации для трех описанных методов: K-Means, Mean Shift, DBSCAN.

ГЛАВА 5

СЕГМЕНТАЦИЯ НА ОСНОВЕ АКТИВНОГО КОНТУРА

Сегментация изображений на основе активного контура (Active Contour Segmentation, также известна как Snake Segmentation) – это один из методов компьютерного зрения, который позволяет выделить объект на изображении путем создания замкнутой кривой (контура), которая охватывает границы объекта.

Основная идея этого метода заключается в том, чтобы определить контур, описывающий границу объекта, который нужно выделить на изображении. Контур представляется в виде замкнутой кривой или поверхности, и он перемещается на изображении, пока не достигнет определенного состояния. Это состояние достигается путем минимизации энергии контура, которая определяется с использованием различных функционалов.

Активный контур состоит из двух основных компонентов: модели формы и модели энергии. Модель формы представляет собой математическое описание формы объекта, который нужно выделить. Модель энергии определяет, какие точки на изображении должны быть притянуты или отталкиваться от контура, чтобы определить его конечную форму. Модель энергии также может включать различные функционалы, такие как градиент яркости, кривизну контура и силу натяжения.

Алгоритм активного контура состоит из нескольких этапов.

1. Инициализация. На изображении выбирается начальное положение контура, которое может быть задано пользователем или автоматически.

2. Вычисление энергии. Контур представляется в виде математической кривой, например сплайн-кривой, которая устанавливается на изображении вокруг объекта. Далее вычисляется энергия контура, которая состоит из двух частей: внутренней энергии и внешней энергии.

3. Обновление контура. Контур обновляется путем изменения его формы и положения.

4. Остановка алгоритма. Алгоритм останавливается, когда контур перестает изменять свою форму и расположение, это означает, что он соответствует границам объекта.

Внутренняя энергия вычисляется на основе свойств контура, таких как его длина, кривизна и так далее. Цель внутренней энергии – сделать контур гладким и замкнутым.

Внешняя энергия вычисляется на основе свойств изображения, таких как яркость, текстура, градиент и так далее.

Преимущества метода активного контура включают:

- способность выделять объекты со сложными формами и текстурами;
- гибкость и адаптивность алгоритма к изменениям контура объекта;
- возможность автоматической сегментации без необходимости ручной настройки параметров алгоритма.

Однако метод активного контура также имеет некоторые недостатки:

- сложность подбора параметров алгоритма, которые могут сильно влиять на результат сегментации;
- чувствительность к начальному положению контура, что может привести к ошибочной сегментации;
- недостаточная точность для выделения мелких деталей объекта.

Ниже приведем реализацию данного метода сегментации с помощью библиотеки `skimage`.

```
import numpy as np
import cv2
import skimage.segmentation as seg

# Загрузите изображение
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

def circle_points(resolution, center, radius):
    """
    Генерация точек, находящихся на границе круга.
    """
    radians = np.linspace(0, 2*np.pi, resolution)
    # полярные координаты
    c = center[1] + radius*np.cos(radians)
    r = center[0] + radius*np.sin(radians)
    return np.array([c, r]).T

# Исключение последней точки, поскольку замкнутый
круг не должен иметь повторяющихся точек
points = circle_points(resolution=300, center=[170,
170], radius=110)[: -1]
```

```
# Сегментация активного контура
snake = seg.active_contour(image, points, alpha=0.2,
beta=0.5)

# Визуализация
fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(7, 7))
ax.imshow(image, cmap='gray')
ax.axis('off')
ax.plot(points[:, 0], points[:, 1], '--r', lw=3)
ax.plot(snake[:, 0], snake[:, 1], '-g', lw=3);
```

В данной программе функция `circle_points` вычисляет координаты x и y точек на границе круга. Так как в нашем случае – `resolution=300`, она вычислит 300 таких точек. Затем алгоритм сегментирует объект на изображении, подгоняя замкнутую кривую к краям объекта. Можно настроить параметры под названием `alpha` и `beta` функции `seg.active_contour()`. Более высокие значения `alpha` заставляют кривую сокращаться быстрее, тогда как `beta` делает контур более гладкой. На рисунке 13 приведен пример выделения контура объекта.



Рис. 13. Выделение объекта на основе активного контура.
Красный – начальное положение контура, зеленый – результат

Контрольные вопросы

1. Что такое метод активного контура (Snake Segmentation)?
2. Какова основная цель метода активного контура?
3. Какие этапы включает алгоритм активного контура?
4. Какие преимущества и недостатки связаны с методом активного контура?

Лабораторная работа

1. Примените метод активного контура к объектам на изображении с различными начальными формами контура (прямоугольник, треугольник, эллипс и т. д.).
2. Проведите исследование по оптимизации параметров α и β для конкретного изображения.

ЛИТЕРАТУРА

1. Обнаружение объектов методом Оцу. – URL: <https://habr.com/en/articles/112079/>
2. Метод активных контуров для сегментации изображений. – URL: <https://imaging.cs.msu.ru/files/courses/varmethods2022/varmethods2022-book-2.pdf>
3. Предварительная обработка изображений. – URL: <https://api-2d3d-cad.com/processingimages/>
4. Python OpenCV – морфологические операции. – URL: <https://bestprogrammer.ru/programmirovanie-i-razrabotka/python-opencv-morfologicheskie-operatsii>
5. Введение в сегментацию изображений с помощью кластеризации К-средних. – URL: <https://machinelearningmastery.ru/introduction-to-image-segmentation-with-k-means-clustering-83fd0a9e2fc3/>
6. Обзор алгоритмов сегментации. – URL: <https://habr.com/ru/companies/intel/articles/266347/>
7. Осваиваем компьютерное зрение – 8 основных шагов. – URL: <https://habr.com/ru/articles/461365/>
8. Семантическая сегментация: краткое руководство. – URL: <https://neurohive.io/ru/osnovy-data-science/semantic-segmentation/>

*Электронное учебное издание
сетевого распространения*

Тумаков Дмитрий Николаевич
Каюмов Зуфар Дамирович
Егорчев Антон Александрович
Латыпов Ильдар Ирекович

АЛГОРИТМИЧЕСКИЕ МЕТОДЫ СЕГМЕНТАЦИИ ИЗОБРАЖЕНИЙ

Учебное пособие

Корректор
Л.И. Хасанова

Компьютерная верстка
М.А. Ахметова

Дизайн обложки
М.А. Ахметова

Подписано к использованию 29.11.2023.
Гарнитура «Times New Roman».
Заказ 39/10.

Издательство Казанского университета
420008, г. Казань, ул. Профессора Нужина, 1/37
тел. (843) 206-52-14 (1704), 206-52-14 (1705)